

CS2106 Cheatsheet, AY26/27 S1

Zhao Wu and Tien Cheng

August 22, 2026

1 Introduction

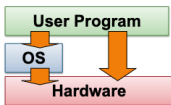
What is OS?

OS is a program that acts as an **intermediary** between a **computer user** and the **computer hardware**. (Simplified definition)

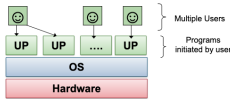
- E.g.
 - Windows, Mac OS, Linux, Solaris, FreeBSD
 - iOS, Android
 - PSS, Xbox, Nintendo
 - Smart TV, Smart Watch

Brief History of OS

- First computer:
 - OS Type:** No OS
 - Program directly interacts with hardware.
 - Reprogram by physically changing configuration of hardware (cable, switches, punched paper type)
 - Advantage:**
 - Minimal overhead due to OS
 - Disadvantage:**
 - Not portable (to port, need manually rewrite the program)
 - Inefficient use of computer?
 - E.g. Electronic Numerical Integrator And Computer (ENIAC), Harvard Mark I
- Mainframes
 - Characteristics:**
 - No interactive interface (program via paper tape, magnetic tape, punch card)
 - Batch Processing Only
 - OS Type:** Batch OS
 - User still interact with hardware directly
 - Additional information for OS (e.g. resource required, job specification)
 - Advantage:**
 - Execute user program (ie Job) one at a time
 - Disadvantage**
 - Simple batch processing is inefficient as CPU idle when I/O.
 - Multiprogramming is absent.
 - E.g. IBM 360



- Time-Sharing OS
 - OS Type:** Time-Sharing OS
 - Advantage:**
 - Allow multiple users to interact with machine using terminals (teletypes)
 - User job scheduling (illusion of concurrency)
 - OS manages CPU time, memory and storage
 - Virtualization of hardware (each program executes as if it has all the resources to itself)
 - E.g. Apple II PC, IBM PC



- Personal OS
 - Windows model:
 - Single user at a time, but possibly more than 1 user can access
 - General time-sharing model
 - Unix model:
 - One user at workstation but other users can access remotely

- General time sharing model

1.1 Motivation of OS

1. Abstraction

- Hide low-level details and present higher-level functionality to user
- Motivation:
 - Large variation in hardware configuration, but same hardware has well defined functionality. (E.g. rotation speed for hard disk varies but it can stores and retrieve information)
 - Efficiency, Programmability and Portability

2. Resource Allocator

- Manage all resources (CPU, Memory, IO) and Arbitrate potentially conflicting request for efficient and fair resource use
- Motivation:
 - Program requires multiple hardware resources to run.
 - Multiple programs should run simultaneously for better utilization

3. Control Program

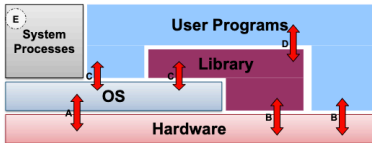
- Control execution of programs to:
 - Prevent errors and improper use of computer
- Motivation:
 - Program may "misuse" the computer
 - Bugs (accident), Virus, Malware (malicious)
 - Multiple user can share the computer
- Security, isolation and protection

4. Portability and Efficiency

1.2 OS Structures

High-Level View of OS

- OS is a software that runs in **Kernel Mode** (i.e. Direct access to All hardware resources)
 - Can't use system calls in kernel code
 - Can't use normal libraries and I/O
- Other software operates in **User Mode** (i.e. Limited/Controlled access to hardware resources)

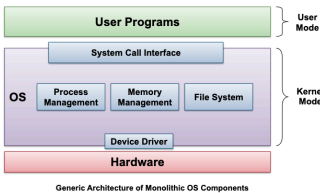


- A:** OS executing machine instructions
- B:** Normal machine instructions executed (program/library code)
- C:** Calling OS using **system call interface**
- D:** User program calls library code
- E:** System processes
 - Provide high-level services, usually part of OS

OS Structures

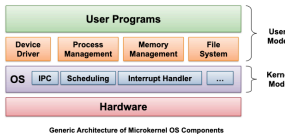
1. Monolithic

- Big program (e.g. Linux source-code)
- If Kernel fails, BSOD
- Better performance



2. Microkernel

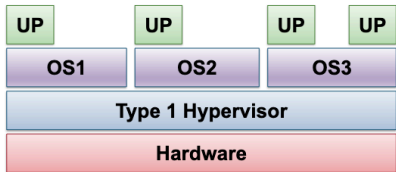
- Smaller and cleaner abstraction
- Provide basic and essential facilities:
 - Inter-Process Communication
 - Address space management
 - Thread management
- Higher-level OS services are run outside of the kernel, using IPC to communicate.
- Lower performance



1.3 Virtual Machines

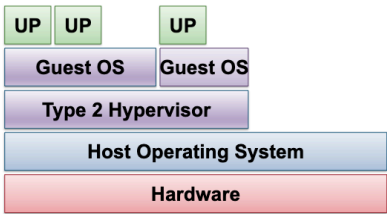
Virtual Machine emulates the underlying hardware, created and managed by **Hypervisor** (aka **Virtual Machine Monitor** (VMM))

1. Type 1 Hypervisor



- Type 1 hypervisor:**
 - AKA bare-metal hypervisor
 - Provides individual *virtual machines* to guest OSES
 - E.g., IBM VM/370, VMware ESXi

2. Type 2 Hypervisor



- Type 2 hypervisor**
 - Runs in host OS
 - Guest OS runs inside Virtual Machine
 - e.g., VMware Workstation, VirtualBox, QEMU

2 Process Abstraction

2.1 Process Context

A **process** (or task or job) is the OS abstraction for a running program.

To describe the execution of a program, the system needs to track:

Layer	Component	Stores
Memory	Instructions	program code
	Data	globals + static

Layer	Component	Stores
	Stack	frames (calls + locals)
	Heap	dynamically allocated data
	GPRs	temporary data
	PC	next instruction address
Hardware	SP	top of stack
	FP	base of current frame
OS	PID	unique process ID
	Process State	current process state

2.2 Stack Memory & Function Calls

Stack Memory

- The **stack** is a region of memory used to support function calls.
- Each function invocation pushes a new stack frame onto the stack, while each function return pops the top stack frame.
- Each stack frame contains (CS2106 convention, caller-pushed first):

Component	Who	Why
Arguments (only those that don't fit in registers)	Caller	so the callee can read its inputs
Return address (saved PC)	Caller	return to the right instruction after the callee returns
Saved old FP	Callee	restore the caller's FP before returning
Saved old SP	Callee	restore the caller's SP (pop this frame)
Saved callee-saved GPRs	Callee	only GPRs the callee modifies; protect the caller's values across the call
Local variables	Callee	space for the callee's own variables

- The stack pointer (SP) points to the top of the stack
- The frame pointer (FP) points to the base of the current stack frame

Function Calls

2.3 Heap Memory

2.4 Process States